EEL 5764: Computer Architecture

Lectures 7–8: Quantitative Architectural Evaluation

Sandip Ray

Warren B. Nelms Endowed Professor
Department of Electrical & Computer Engineering
University of Florida | Office: MALA 4131

Required Reading: Hennessy & Patterson Sections 1.8 & 1.9, Patterson & Hennessy Chapter 1

Workload Selection: The Benchmark Spectrum

THE SPECTRUM OF EVALUATION

Workload Definition: The set of programs run on a computer. An architect's performance claims are only as valid as the workload selected for evaluation.

- **Real-World Applications:** Unmodified user programs (e.g., GCC compiler, Photoshop, database engines). These provide the highest accuracy of actual performance but are difficult to port and run during simulation.
- **Kernel Algorithms:** Small, key computational kernels extracted from real-world applications (e.g., Matrix Multiply, Fast Fourier Transform, AES Encryption). Excellent for isolating specific hardware units.

TOY CODES & MICROBENCHMARKS

- **Toy Programs:** Very small, simple programs (e.g., Sieve of Eratosthenes, QuickSort, BubbleSort). They are self-contained and easy to compile but do not reflect real-world memory access and instruction mix complexities.
- **Synthetic Benchmarks:** Artificial programs designed to match a target instruction mix (e.g., Dhrystone for integer logic, Whetstone for floating-point calculations). They are highly portable but can be easily exploited by optimizing compilers.
- **Workload Representativeness**: Graduate research projects must explicitly justify workload selection. Never assume a single toy code represents general system performance.

The Pitfalls of Synthetic Benchmarks

THE FAILURE OF DHRYSTONE & WHETSTONE

- **The Compiler Exploitation Trap**: Synthetic benchmarks contain no real computation and do not process meaningful input data. This makes them highly vulnerable to optimizing compilers.
- **Dead Code Elimination**: Because the results of loop calculations in synthetic benchmarks are often never utilized, modern optimizing compilers can detect them as dead code and completely eliminate the loop during compilation.
- **False Performance Claims:** In the 1980s and 1990s, compilers routinely optimized away major portions of Dhrystone, showing a 10x 'performance boost' on hardware that actually ran real applications at the exact same speed.

TYPICAL EXAM PREPARATION CASE

- An x86 processor executes a synthetic loop 10,000 times to calculate a value that is never written to disk or memory. An unoptimized compiler schedules 50,000 instructions.
- **An optimized compiler** with dead code elimination enabled completely strips the loop, reducing the instruction count to exactly 3 instructions. The synthetic metric shows a 16,666x speedup.
- **The Quantitative Moral:** Always compile benchmarks with production-grade optimization flags (e.g., -O3) and verify that the compiler has not optimized away the core workload.

Industry Standards: The SPEC CPU Suites

SPEC CPU SUITE CURATION

- **SPEC (**Standard Performance Evaluation Corp): A non-profit consortium founded to establish realistic, standardized benchmarks for modern high-performance processors.
- **Real-World Source Code**: Curated from real, heavily utilized open-source applications (e.g., GCC compiler, Perl interpreter, quantum physics simulators, fluid dynamics solvers).
- **Regular Updates:** SPEC periodically updates its suites (e.g., SPEC CPU2006, CPU2017) to prevent compiler-specific tuning and to ensure workloads reflect modern code footprints (like larger cache working sets).

FORMULATING THE SPECRATIO

- **Normalized Execution Time:** SPEC measures execution time. To aggregate diverse applications, it normalizes times against a standard reference machine (e.g., an older UltraSPARC processor).
- The SPECratio Equation:
  - SPECratio_i = Execution_Time_Reference_i / Execution_Time_Candidate_i
- **High SPECratio = High Performance**: A SPECratio of 12.0 means the candidate machine executes that specific benchmark 12.0 times faster than the reference system.
- **work and evaluation:** SPEC ratios are multiplicative speedups, not raw rates.

Summarizing Performance: The Aggregation Problem

THE MATHEMATICAL DILEMMA

- **The Problem:** A candidate processor executes a benchmark suite containing 30 different applications. How do we aggregate these 30 distinct SPECratios into a single, representative metric?
- **The Three Candidates**: Architects choose among the Arithmetic Mean, Geometric Mean, and Harmonic Mean depending on the underlying data dimension.
- **Arithmetic Mean (AM):** Sum of values divided by count. Extremely intuitive, but mathematically invalid for normalized ratios.

THE FORMULAS IN CONTEXT

- **Arithmetic Mean of Execution Times:** Directly proportional to total workload time on a single machine. Mathematically valid if all programs run an equal number of times.
- **Geometric Mean (GM) of Speedups:** Evaluates normalized ratios without bias. Used exclusively by SPEC.
- **Harmonic Mean (HM) of Rates: U**sed when summarizing rate-based metrics (e.g., MIPS, bandwidth, transactions per second).

The Golden Rule: Never apply the Arithmetic Mean to speedup ratios or execution rates. It introduces systemic mathematical bias.

Aggregating Ratios: The Geometric Mean

THE GEOMETRIC MEAN ADVANTAGE

- **Formulating Geometric Mean:** The n-th root of the product of n data points:

  Geometric_Mean = (Product_{i=1}^{n} Ratio_i)^(1/n)

- **Reference Machine Independence:** When evaluating relative speedup ratios, the Geometric Mean is the only average that guarantees consistent relative rankings regardless of which reference machine is chosen as the baseline.
- **No Choice Bias:** If Machine X is faster than Machine Y using reference A, it will remain faster using reference B. The Arithmetic Mean fails this property.

THE FAILURE OF ARITHMETIC AVERAGE

- **The Choice-Dependence Trap:** If you normalize execution times to Machine A, and take the Arithmetic Mean, Machine A can appear faster than Machine B.
- **Reversing the Results:** If you take the exact same raw execution times, normalize them to Machine B, and take the Arithmetic Mean, Machine B can suddenly appear faster than Machine A!
- **SPEC Standard:** To maintain absolute scientific objectivity, SPEC mandates the Geometric Mean for aggregating all SPECratios.

Proof of Reference Independence

THE STEP-BY-STEP ALGEBRAIC PROOF

- 1. Let there be two candidate machines, X and Y, running a suite of n benchmarks.
- 2. Let Ref_A be reference machine A. The SPECratio for machine X on benchmark i normalized to Ref_A is X_i / A_i.
- 3. The aggregated Geometric Mean speedup of X relative to Ref_A is: GM_X_A = [Product_{i=1}^{n} (X_i / A_i)]^(1/n)
- 4. Similarly, for Y relative to Ref_A: GM_Y_A = [Product_{i=1}^{n} (Y_i / A_i)]^(1/n)
- 5. Now, examine the ratio of their aggregated performances: GM_X_A / GM_Y_A = [Product_{i=1}^{n} (X_i / A_i)]^(1/n) / [Product_{i=1}^{n} (Y_i / A_i)]^(1/n)
- 6. Since the n-th root of a product can be factored, the reference term A_i cancels out perfectly: GM_X_A / GM_Y_A = [Product_{i=1}^{n} (X_i / Y_i)]^(1/n) = GM_X_Y

Architectural Moral: The relative performance ratio between X and Y is completely independent of the reference machine A. The reference terms cancel. If we used the Arithmetic Mean, the summation would prevent this factoring and cancelation, locking the result to the baseline choice.

Aggregating Rates: The Harmonic Mean

UNDERSTANDING EXECUTION RATES

- **Rate-Based Metrics:** Expressed as work per unit of time (e.g., Millions of Instructions Per Second (MIPS), Megabytes per second (MB/s), Transactions per Second).
- **The Harmonic Mean Formula:** The reciprocal of the Arithmetic Mean of the reciprocals:
  - Harmonic_Mean = n / Sum_{i=1}^{n} (1 / Rate_i)

  The Dynamic Consistency Rule: When aggregating rates, the Harmonic Mean is the only average that maintains direct consistency with total execution time.

THE CRITICAL METRIC PITFALL

- **The Arithmetic Mean Rate Lie:** If you average the MIPS rate of two programs using the Arithmetic Mean, the resulting 'average MIPS' will correspond to a total execution time that is physically impossible on the actual machine.
- **Skewed towards High Rates:** The Arithmetic Mean is heavily skewed by extremely fast benchmarks that run in negligible time, completely hiding the bottlenecks of slow benchmarks.
- **Summary Principle:** Aggregating execution times -> Arithmetic Mean. Aggregating normalized speedups -> Geometric Mean. Aggregating work rates -> Harmonic Mean.

Proof of HM vs. AM Disparity

LINKING RATES TO TIME

- 1. Let a benchmark suite execute a constant amount of work W (e.g., instructions) on each of n programs.
- 2. Let the execution rate for program i be: R_i = W / T_i => T_i = W / R_i.
- 3. The total execution time for the entire suite is the simple sum of individual times: T_total = Sum_{i=1}^{n} T_i = Sum_{i=1}^{n} (W / R_i)
- 4. We want to find the true average rate R_avg that satisfies: R_avg = (Total Work) / (Total Time) = (n * W) / T_total
- 5. Substitute T_total into the equation: R_avg = (n * W) / [Sum_{i=1}^{n} (W / R_i)]
- 6. Factoring out and canceling the constant Work term W: R_avg = n / [Sum_{i=1}^{n} (1 / R_i)]

Architectural Moral: The true average execution rate (R_avg) is exactly the Harmonic Mean of the individual rates. A simple Arithmetic Mean of the rates would yield an invalid speed projection, as addition cannot reside in the denominator of a time equation.

Quick Challenge: Aggregating SPEC Performance

THE DUAL-MACHINE SPEC COMPARISON

- Scenario: Two candidate processors, Machine X and Machine Y, are evaluated using three SPEC benchmarks normalized against a reference SPARC station:
  - - Benchmark 1: SPECratio of Machine X = 1.0 | SPECratio of Machine Y = 10.0
  - - Benchmark 2: SPECratio of Machine X = 1.0 | SPECratio of Machine Y = 0.1
  - - Benchmark 3: SPECratio of Machine X = 8.0 | SPECratio of Machine Y = 5.0
- Your Tasks:
  - 1. Calculate the Arithmetic Mean of the SPECratios for both machines.
  - 2. Calculate the Geometric Mean of the SPECratios for both machines.
  - 3. Compare the outcomes: Which machine 'wins' under each mean? Discuss the danger of using the Arithmetic Mean to market hardware.

*Solution Hint: AM_X = (1+1+8)/3 = 3.33 | AM_Y = (10+0.1+5)/3 = 5.03 (Y wins!). GM_X = (1*1*8)^(1/3) = 2.0 | GM_Y = (10*0.1*5)^(1/3) = 1.71 (X wins!). Y wins the AM because Benchmark 1 is 10x, but Y is actually 10x slower on Benchmark 2, which GM captures properly.*

Multi-Stage Amdahl's Law

BEYOND SINGLE-PART SPEEDUP

- **Review of Single-Part Speedup:** Amdahl's Law determines the overall performance gain from improving a single, localized block of code.
- **The Reality of Complex SoCs: M**odern heterogeneous processors do not implement a single enhancement. They deploy multiple, disjoint accelerators simultaneously (e.g., SIMD vector units, caches, floating-point units).
- The Generalized Multi-Stage Equation:
  - Speedup_overall = 1 / [(1 - Sum_{i=1}^{M} f_i) + Sum_{i=1}^{M} (f_i / s_i)]
- Where:
  - - f_i: Fraction of the baseline execution time enhanced by unit i.
  - - s_i: The local speedup factor of that specialized unit.

THE LAW OF MULTIPLE BOTTLENECKS

- **The Interaction Penalty**: Improving one section of the processor (like the floating-point unit) shifts the performance bottleneck entirely onto the remaining unenhanced sections.
- **The Sequential Floor:** The term (1 - Sum f_i) represents the unenhanced, serial fraction of execution time. It acts as an absolute ceiling on system performance.
- **Balanced Optimization**: High-performance design requires optimizing multiple, disjoint bottlenecks concurrently, as single-point acceleration yields rapid diminishing returns.

Example: Multi-Stage Optimization

THE MULTI-VARIABLE ARCHITECTURE ALLOCATION PROBLEM

- Scenario: A baseline CPU spends its execution cycle time on three disjoint operations:
  - - Floating-point instructions (FP): f_FP = 30% of execution time.
  - - Main Memory accesses (Mem): f_Mem = 40% of execution time.
  - - Branch/Control Flow instructions (Br): f_Br = 15% of execution time.
- Your Task: Evaluate the following compound microarchitectural proposals relative to the baseline machine:
  - - Proposal A (Accelerated CPU): Adds a wide SIMD unit that accelerates FP by 5x (s_FP = 5), and a prefetcher that accelerates Mem by 2x (s_Mem = 2).
  - - Proposal B (Specialized SoC): Adds a custom matrix engine accelerating FP by 10x (s_FP = 10) and a branch predictor accelerating Br by 3x (s_Br = 3).
  - Calculate the overall speedup for both proposals. Which design should your team choose?
  - *Solution Key: Unenhanced fraction = 1 - 0.30 - 0.40 - 0.15 = 0.15.*
  - *Speedup_A = 1 / [0.15 + (0.30/5) + (0.40/2) + 0.15] = 1 / [0.15 + 0.06 + 0.20 + 0.15] = 1 / 0.56 = 1.78x*
  - *Speedup_B = 1 / [0.15 + (0.30/10) + 0.40 + (0.15/3)] = 1 / [0.15 + 0.03 + 0.40 + 0.05] = 1 / 0.63 = 1.58x*
  - *Verdict: Choose Proposal A! Even though B has a massive 10x FP speedup, A delivers better performance because it addresses the larger memory bottleneck (40% vs 30%).*

Amdahl's Law in Core Allocation

THE AREA VS. PERFORMANCE TRADEOFF

- **The Silicon Area Currency:** In modern chip design, physical die area is a fixed, non-yielding cost. We measure hardware complexity in Base Core Equivalents (BCE).
- **Pollack's Rule:** Within a given process technology node, microarchitectural performance scales roughly as the square root of physical die area and complexity:
  - Performance_Core = k * \sqrt{Area_Core}
- **The Design Split:** A 1-BCE core provides 1.0 unit of performance. A wide 4-BCE out-of-order core provides only 2.0 units of performance.

ASYMMETRIC CORE ECONOMIC SUMMARY

- **The Homogeneous Multi-Core Trap:** Spending a 16-BCE area budget on sixteen (16) simple 1-BCE cores is ideal for highly parallel software (f -> 1.0), but runs single-threaded code at a crawl (speedup = 1.0).
- **The Asymmetric Multi-Core Escape:** Spending the 16-BCE budget on one massive 4-BCE performance core (delivering 2.0x single-thread speed) paired with twelve (12) simple 1-BCE cores.
- **Amdahl's Speedup validation:** The single massive core accelerates the serial bottleneck (1-f), while the small cores handle the parallel thread segments.

Dynamic Power vs. Active Energy

THE PHYSICAL SEPARATION

- **Power (Watts):** The rate of energy consumption per unit of time: P = dE / dt. Determines physical heat dissipation, voltage drop, and cooling demands.
- **Energy (Joules)**: The total physical work performed over an execution duration: Energy = Power x Time. Determines battery life and electrical cost.

The Trade-off Conflict: A designer can lower power by slowing down the processor clock frequency, but this stretches execution time, keeping total energy consumption flat.

THE METRIC TARGETS

- **Peak Power (Watts):** Limits the peak current drawn from the supply grid. High peak power triggers voltage droop (L * di/dt noise), crashing circuits.
- **Thermal Design Power (TDP):** The maximum heat a cooling solution (fans, heat sink) is physically rated to dissipate. Exceeding TDP causes thermal throttling.
- **Total Energy (Joules):** The true metric for battery-powered client devices and large cloud datacenters where electricity bills dominate operating costs.

Thermal Design Power & di/dt Noise

THERMAL DESIGN POWER (TDP)

- **TDP Definition:** The sustained power draw that a chip is physically guaranteed to maintain without exceeding its junction thermal limits (typically 100°C for silicon).
- **Cooling Budgets:** Server systems deploy aggressive air/liquid cooling rated up to 350W TDP. Fanless mobile chassis are strictly capped at 5W to prevent burning skin.
- **Thermal Throttling:** When localized temperature sensors detect hotspot peaks, internal microcode automatically throttles voltage and frequency to protect the silicon.

POWER GRID AND di/dt NOISE

- **The di/dt Challenge**: High-speed transistors switching states cause rapid changes in current draw over nanosecond scales (high di/dt).
- **Inductive Voltage Droop:** Physical chip packages have parasitic inductance L. Rapid current spikes trigger an inductive voltage droop:
  - V_drop = L * (di / dt)
- **The Noise Penalty:** If voltage droops below the noise margin floor, logic gates misinterpret bits, causing silent data corruption or hard crashes.
- **Mitigation:** Integrated decoupling capacitors acting as local charge reservoirs.

Dynamic Voltage and Frequency Scaling

THE PHYSICS OF SCALING

- **DVFS Concept:** Dynamically adjusting a processor's operating voltage and clock frequency concurrently to match the instantaneous workload demands.
- **Cubic Power Reduction:** Because dynamic power scales as $V^2 * f$, and operating frequency is linearly proportional to voltage ($f \sim V$), reducing voltage by 20% yields a ~50% reduction in dynamic power!
- **Workload Profiling:** During idle background phases, the operating system kernel throttles the hardware state down, ramping up to turbo frequencies only during active bursts.

THE DVFS WALL

- **The Sub-Micron Leakage Fl**oor: Operating voltage cannot scale down indefinitely. Transistors require a minimum threshold voltage V_{th} (~0.7V-0.8V) to switch functional states.
- **Slowing Transistors**: As supply voltage V approaches the threshold floor, transistor gate delay increases exponentially, destroying operating frequencies.
- **The DVFS Breakdown:** Modern 3nm/2nm process nodes operate very close to this voltage margin, meaning DVFS no longer provides the massive generational power savings of the past.

Evaluating Efficiency: Energy-Delay Product

THE MERIT OF BALANCED METRICS

- **The Performance Rate Illusion**: Designers can achieve high performance by running hardware at extreme voltages and frequencies, but the dynamic power scales cubically, burning battery life.
- **The Low-Power Illusio**n: Alternatively, a designer can run a chip at microwatts by lowering the clock to 1 MHz, but execution delay stretches to hours.
- **Energy-Delay Product (EDP):** A unified architectural metric that evaluates performance and energy efficiency simultaneously:
  - EDP = Energy_Consumed x Execution_Delay

INTERPRETING LOW EDP VALUES

- **Smaller is Better: A lo**wer EDP indicates a superior microarchitecture that completes tasks quickly while conserving battery power.
- **Weighting Performance:** Since Execution Delay is included, EDP penalizes designs that slow execution to save power.

Evaluative Scope: EDP is the standard metric used in peer-reviewed research and industrial design specifications (like Intel or ARM cores) to prove real structural efficiency.

Voltage Invariance and Energy-Delay Squared

THE VOLTAGE SCALING BIAS

- **The EDP Bias:** While EDP is a solid balanced metric, it remains biased by voltage scaling. A designer can frequently lower EDP simply by tuning the hardware voltage down.
- **Faking Architectural Gains**: Voltage scaling is a physical technology trick, not an architectural layout change. We need a metric that isolates true microarchitectural gains.
- **Energy-Delay Squared (ED2P):** Formulated to neutralize the effect of basic physical voltage scaling:
  - ED^2P = Energy x Delay^2

PROOF OF VOLTAGE INVARIANCE

- **Let us scale voltage:** Delay (D) scales roughly as 1/V. Energy (E) scales roughly as V^2.
- Substitute these scaling factors into EDP:
  - EDP = E x D => V^2 x (1/V) = V (Dependent on V!)
- Now, substitute into ED2P:
  - ED^2P = E x D^2 => V^2 x ($1/V^2$) = 1 (Voltage-Invariant!)
- **Architectural Utility:** ED2P isolates pure microarchitectural innovation (caches, schedulers) from physical voltage adjustments.

Silicon Economics: NRE Costs

NON-RECURRING ENGINEERING (NRE)

- **NRE Definition:** The upfront, one-time engineering expenses required to design, verify, and prepare a microchip layout for high-volume manufacturing.
- **Design and Verification Team**s: Modern high-performance processors require hundreds of engineers working for 2-3 years. Salary costs routinely exceed $100 Million USD.
- **Electronic Design Automation** (CAD): Specialized software tools (Synopsys, Cadence) for logical synthesis, routing, and physical layout check license fees are massive.

THE PHYSICAL MASK SET PENALTY

- **The Lithography Masks:** Physical quartz plates used during optical photolithography to print circuit features onto silicon wafers.
- **The 3nm Mask Penalty:** At legacy 28nm nodes, a complete physical mask set cost ~$1 Million USD. At modern 3nm nodes, a mask set exceeds $15 Million USD!
- **Amortization Bound:** High NRE costs explain why custom chip startups are rare. You must sell millions of chips to amortize the upfront NRE cost.

Silicon Economics: Recurring Costs

RECURRING SILICON COSTS

- **Recurring Costs:** The continuous, per-unit expenses incurred for manufacturing each individual physical microprocessor.
- **Raw Wafer Processing:** Slicing silicon ingots into 300mm wafers and processing them through photolithography. A TSMC 3nm wafer is fixed at ~$20,000 USD.
- **Silicon Testing:** Functional validation of raw printed dies using automated probe testers prior to physical dicing and packaging.

PACKAGING & THE YIELD PENALTY

- **Physical Packaging**: Encasing the silicon die in ceramic/plastic, routing pins to package pads. Advanced multi-chiplet packaging (CoWoS) adds massive cost.
- **The Final Test Yield Penalty**: If a wafer has low die yield (e.g., only 30% of dies functional), the cost of the raw wafer is divided among fewer functional units, exponentially driving up individual chip costs.
  - The Cost of IC Equation: Cost_IC = (Cost_Die + Cost_Test + Cost_Pkg) / Final_Test_Yield

Design Comparison: Core Width vs. Frequency

THE QUANTITATIVE ARCHITECTURAL TRADEOFF SCENARIO

- Design Objective: Build a processor core under a strict, flat dynamic power envelope of P_max = 30 Watts on a 5nm process node.
- The Engineering Team presents two mutually exclusive microarchitectural designs:
  - - Design A (Wide & Slow): A wide, out-of-order execution core with high active capacitance C_A = 15 nF. It achieves an excellent, low Cycles Per Instruction of CPI_A = 0.80. To operate stably, it requires a supply voltage of V_A = 0.90 V.
  - - Design B (Narrow & Fast): A narrow, deeply-pipelined core with lower active capacitance C_B = 10 nF. It has a higher average Cycles Per Instruction of CPI_B = 1.20 due to pipeline stall cycles. It operates stably at V_B = 1.10 V.
  - **Your Challenge:** Calculate the maximum operating frequency for both designs under the strict 30W power limit, and determine which design delivers superior performance (lowest execution time per program). Assume the instruction count (IC) and activity factor are constant.
  - Use the Dynamic Power Equation: P_dynamic = C * V^2 * f => f_max = P_max / (C * V^2)

Solving the Width vs. Frequency Tradeoff

THE MATHEMATICAL DERIVATION

- 1. Solve for maximum operating frequencies under the flat P_max = 30W envelope:
  - - Design A: f_max_A = 30 W / (15 * 10^-9 F * (0.90 V)^2) = 30 / (15 * 10^-9 * 0.81) = 2.47 GHz
  - - Design B: f_max_B = 30 W / (10 * 10^-9 F * (1.10 V)^2) = 30 / (10 * 10^-9 * 1.21) = .2.48 GHz
- 2. Formulate CPU Execution Time ratio using the Iron Law: CPU_Time = IC * CPI / f
  - - Ratio = CPU_Time_A / CPU_Time_B = (IC * CPI_A / f_A) / (IC * CPI_B / f_B)
  - - Ratio = (CPI_A / CPI_B) * (f_B / f_A)
- 3. Substitute the calculated values: Ratio = (0.80 / 1.20) * (2.48 GHz / 2.47 GHz) = 0.667 * 1.004 = 0.669
- 4. Relative Speedup of Design A relative to B: Speedup = 1 / 0.669 = 1.49x

Architectural Verdict: Design A is 49% faster than Design B! Even though both run at practically identical clock frequencies under the flat power envelope, Design A's wider microarchitecture yields a much lower CPI, translating to massive system-level performance gains.

Co-Design & Wiring Complexity

THE PHYSICAL INTERCONNECT LIMIT

- **The Wiring Wall:** As transistor geometries shrank below 45nm, physical gate delays decreased rapidly, but copper/cobalt interconnect wire resistance and capacitance exploded.
- **RC Delay: T**he delay of a physical wire scales quadratically with its length: Delay ~ R_wire * C_wire. Long global wires define the processor's operating cycle limit.
- **Layout Floorplanning:** Microarchitects must physically position execution units to keep high-frequency wiring bypass networks extremely short.

CROSS-LAYER CO-DESIGN ADVANTAGES

- **Overcoming Abstraction Boundaries**: Traditional separation of layers (Application -> Compiler -> ISA -> HW) introduces execution overhead and security vulnerabilities.
- **Domain-Specific Architectures (DSA)**: Custom hardware units (GPUs, NPUs) tailored specifically to accelerate specific high-level algorithms (like AI matrix multiplication).

The Co-Design Moral: Delivering modern performance gains requires co-designing the entire systems stack—compiling custom code directly onto specialized silicon accelerators.